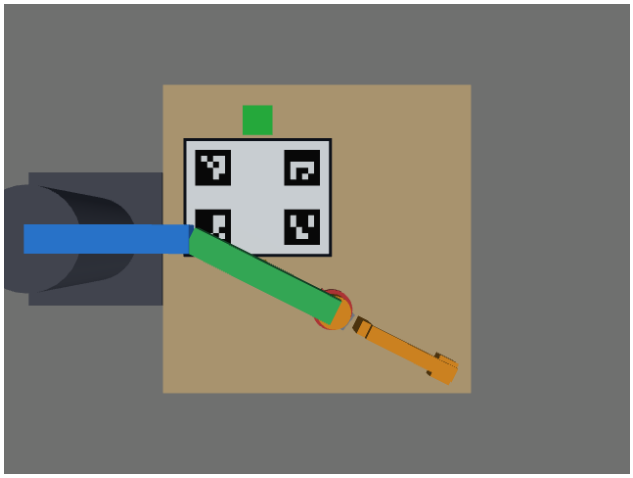
ROS2 + SCARA 开发资料手册

从 URDF、ros2_control、Gazebo 到双相机与手眼标定：所有步骤、公式与实测数据

手册范围	ROS2 构建 / ros2_control / Gazebo / 相机 / ArUco / 手眼标定
验收状态	4 包构建通过 · 控制器 active · 手眼 10 点 8.65 mm / 2.51°
配套代码	python/armlab（N 自由度通用内核，已为 6 轴预留接口）
生成方式	docs/manual/*.md + docs/tools/*.py 自动排版（可一键重生成）
版本 / 日期	v1.0 / 2026-10-16



Gazebo 实测：顶视工作台、四轴 SCARA 与 ArUco 标定板

目录

目录 2

阅读指南 4

第 1 章 环境、包结构与总体架构 4

1.1 本机环境（实测版本） . . . 4

1.2 四个包的分工 . . . 4

1.3 节点 / 话题 / 动作关系 . . . 5

1.4 TF 树 . . . 5

第 2 章 参数复用：从 MuJoCo 场景直接生成 ROS2 资产 7

2.1 为什么要"生成"而不是"手抄" . . . 7

2.2 参数映射表 . . . 7

2.3 相机换算：MuJoCo 的 xyaxes → Gazebo 的 pose . . . 7

第 3 章 URDF / xacro：把机械臂写成机器人描述 9

3.1 连杆与几何 . . . 9

3.2 补算的惯量（为什么必须补） . . . 9

3.3 关节定义 . . . 10

3.4 夹爪几何：28 mm 对 30 mm 的过盈 . . . 10

3.5 ros2_control 接口 . . . 11

第 4 章 控制器配置：每个参数的理由 12

4.1 为什么 update_rate = 1000 Hz . . . 12

4.2 JointTrajectoryController 的关键参数 . . . 12

4.3 控制器启动时序 . . . 12

第 5 章 Gazebo 世界与物理参数 13

5.1 世界清单（由生成器从 MuJoCo 场景写出） . . . 13

5.2 物理参数 . . . 13

5.3 相机分辨率与 GPU 渲染 . . . 13

第 6 章 构建与启动 14

6.1 一键构建 . . . 14

6.2 一个必须处理的坑：入口点位置 . . . 14

6.3 启动 . . . 14

第 7 章 控制链路与计算 15

7.1 运动学代码（与项目同一套公式） . . . 15

7.2 MoveL 轨迹生成（与 MuJoCo 侧同构） . . . 15
7.3 夹爪：Gazebo Classic 的三种方案（实测） . . . 15
7.4 实测的 TCP 序列（8 步，全部到位） . . . 16
第 8 章 联调验证（实测数据） 17
8.1 控制器与状态 . . . 17
8.2 解析 FK □ TF 对照（关键验证） . . . 17
8.3 抓取-放置 . . . 17
8.4 相机 . . . 17
第 9 章 复盘：这次联调踩过的 12 个坑 20
9.6 进程清理的正确写法 . . . 20
9.7 入口点补链接（构建脚本里已自动化） . . . 20
第 10 章 与 MuJoCo 侧的关系 21
10.1 相同点 . . . 21
10.2 不同点（都是平台限制） . . . 21
第 11 章 进阶：视觉闭环、MoveIt2 与力控夹爪（本轮实测） 22
11.1 视觉闭环（cam_top → 台面反投影 → 抓取） . . . 22
11.2 MoveIt2 接入 . . . 23
11.3 力控夹爪插件（真实摩擦抓取尝试） . . . 23
11.4 WSL2 的 GPU 与 Gazebo 渲染（白模与核显问题） . . . 24
11.5 双相机、ArUco 标定板与手眼标定（本轮实测） . . . 25
第 12 章 后续路线 29
附录 A 常用命令 30
附录 B 接口速查 30
附录 C 关键数值 30

阅读指南

这本手册记录 **SCARA-teach-4RPRR 接入 ROS2（Humble）+ Gazebo Classic 11** 的全部步骤：环境、参数复用、URDF、ros2_control、Gazebo 世界、构建启动、控制计算、联调数据与踩坑复盘。所有数字都来自本机实测（WSL2 Ubuntu 22.04）。

三种用法：跟着做（第 2~6 章按顺序执行）；查接口（第 3.4、4、附录 B）；复盘（第 9、10 章）。

与《SCARA 全栈开发资料手册》的关系：那本手册讲"数学模型、MuJoCo、具身智能"；这一本讲"**同一台机械臂如何变成 ROS2 节点，并在 Gazebo 里跑起来**"。两侧共用同一套参数与同一套运动学/轨迹代码。

第 1 章 环境、包结构与总体架构

1.1 本机环境（实测版本）

组件	版本 / 状态	说明
WSL2 发行版	Ubuntu 22.04 (Jammy)	Windows 11 + WSLg（DISPLAY=:0）
ROS2	Humble（/opt/ros/humble）	ros2 --version 正常
Gazebo	Gazebo Classic 11.10.2	gzserver / gzclient
ros2_control	2.54.2	controller_manager / joint_state_broadcaster
ros2_controllers	2.54.0	JointTrajectoryController
gazebo_ros_pkgs	3.9.0	gazebo_ros / gazebo_plugins / gazebo_ros_state
gazebo_ros2_control	0.4.10	**本次补装**（apt install ros-humble-gazebo-ros2-control）
xacro / RViz2	2.1.1 / 11.2.29	URDF 展开与可视化

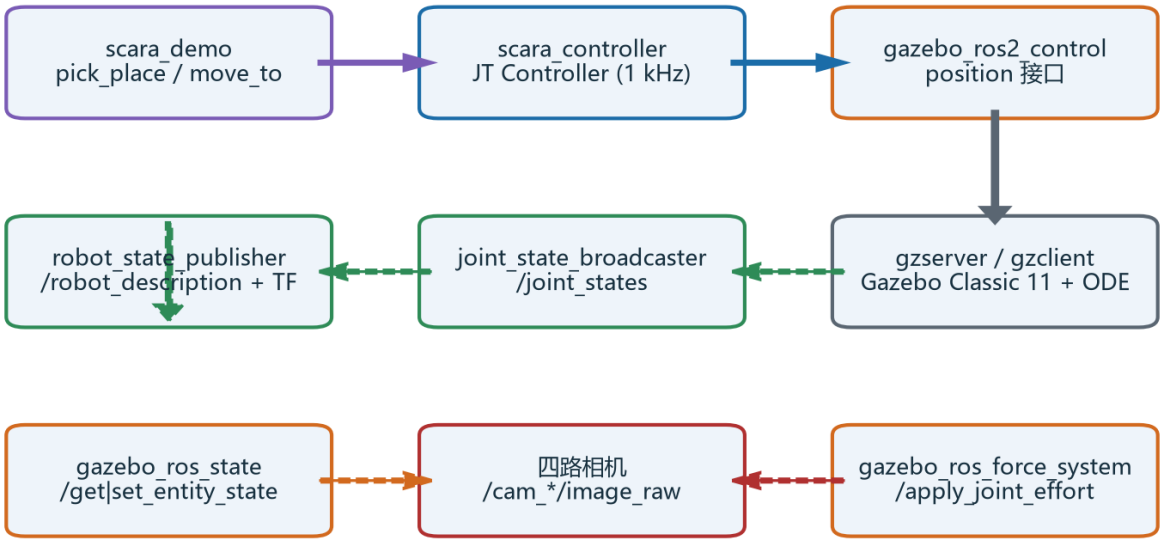
> **为什么先确认版本**：gazebo_ros2_control 与 ros2_control 的接口在 Humble 之后有过变化；0.4.x 配 Humble 是官方组合。版本不匹配时典型症状是"控制器能加载但服务不存在"。

1.2 四个包的分工

包	类型	内容	为什么这样拆
scara_description	ament_cmake	URDF/xacro、生成参数、RViz 配置	描述与运行解耦，RViz/其他工程可直接复用
scara_bringup	ament_cmake	Gazebo 世界、controllers.yaml、两个 launch	"怎么启动"集中在一处，便于换世界/换参数
scara_demo	ament_python	运动学、轨迹、抓取演示、TF 自检、物块监看	任务层逻辑独立，可单独替换为 MoveIt2

包	类型	内容	为什么这样拆
scara_gripper_plugin	Gazebo 插件	力控夹爪进阶实验	与任务代码隔离，ODE 不稳定时可单独禁用

1.3 节点 / 话题 / 动作关系



实线：命令（action / 关节位置）；虚线：状态与服务（joint_states / TF / 图像 / 实体状态 / 力）

图 1-1 ROS2 侧的节点、话题与动作关系（实测运行的图）

核心接口：

- **动作**：/scara_controller/follow_joint_trajectory（control_msgs/action/FollowJointTrajectory）——任务层唯一"动起来"的入口；
- **状态**：/joint_states（1 kHz，joint_state_broadcaster）；
- **描述**：/robot_description（robot_state_publisher，同时发 TF）；
- **相机**：/cam_top/image_raw、/cam_front/image_raw、/cam_eye/image_raw、/cam_wrist/image_raw（各带 camera_info）；
- **Gazebo 服务**：/spawn_entity、/get_entity_state、/set_entity_state、/apply_joint_effort、/clear_joint_efforts。

1.4 TF 树

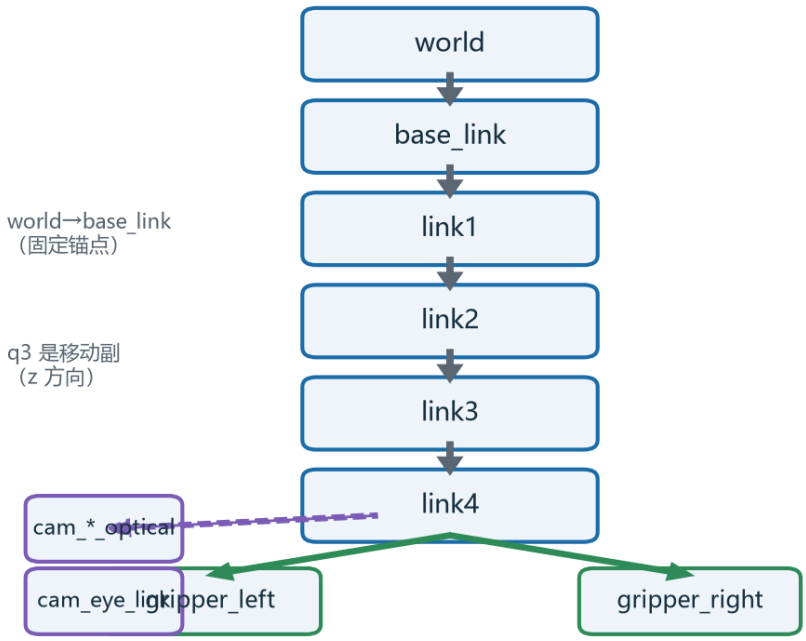


图 1-2 TF 树：world→base_link→link1→…→link4；cam_eye 固定在 link3，cam_wrist 随 link4

关键点：link4 的原点就是项目定义的 **TCP**（与 MuJoCo 里 fk(..., link="link4") 一致）。因此"解析 FK"与"TF 查询"可以直接逐位对比，这是第 8 章自检的基础。

第 2 章 参数复用：从 MuJoCo 场景直接生成 ROS2 资产

2.1 为什么要"生成"而不是"手抄"

项目已有唯一数据源：python/armlab/spec.py（臂长/立柱/行程/质量/惯量/限位）与 python/armlab/scara_scene.py（台面、四腿、物块、放置区、四路相机）。ROS2 侧如果手抄数字，迟早与 MuJoCo 侧漂移。

因此新增生成器 ros2_ws/tools/gen_ros_assets.py：**读 MuJoCo 侧（直接调用 build_scara_pick_place_xml() 得到 MJCF），生成**

1. scara_description/urdf/params.generated.xacro：URDF 的全部数字；
2. scara_bringup/worlds/scara_world.generated.sdf：Gazebo 世界（台面/四腿/放置区/物块/世界相机/物理参数）；
3. outputs/data/ros2_scene_params.json：对照报告（供人工核查）。

2.2 参数映射表

量	MuJoCo 侧来源	ROS2 侧落点	数值
臂长	spec.links[2].pos[0]	xacro a1 / a2	0.225 m
立柱高	spec.links[1].pos[2]	xacro d1	0.300 m
J3 行程	spec.joints[2].upper	xacro stroke	0.200 m
大/小臂质量	links[1]/[2].mass	m1/m2	3.0 / 1.5 kg
质心	links[i].com	com1/com2	0.110 / 0.105 m
惯量	links[i].inertia	I1x/I1yz/I2x/I2yz	见 3.2
夹爪闭合间距	SC.GRIP_GAP_CLOSED	grip_gap_closed	0.028 m
手指行程	SC.GRIP_OPEN	grip_open	0.012 m
台面	SC.TABLE_CENTER/HALF/TOP/THICK	SDF <model name="table">	0.40,0 / 0.26 / 0.10 / 0.02
物块	SC.OBJ_SIZE	SDF target_box	0.03 m
相机	MJCF <camera> 的 pos/xyaxes/fovy	Xacro 属性 + SDF <sensor>	见 2.3

2.3 相机换算：MuJoCo 的 xyaxes → Gazebo 的 pose

MuJoCo 相机朝 **-z**，xyaxes 给出相机的 x/y 轴；Gazebo 相机朝 **+x**。换算步骤（生成器里逐步实现）：

1. 由 x/y 轴求第三轴：z = x × y（右手系）；
2. 取 Gazebo 传感器坐标：前向 f = -z，上向 u = y，左向 l = u × f；
3. 组成旋转矩阵 R = [f, l, u]（列向量），再转成 SDF 的 rpy（R = Rz(yaw)Ry(pitch)Rx(roll)）：

$$\text{pitch} = -\arcsin(R_{20}), \qquad \text{roll} = \operatorname{atan2}(R_{21}, R_{22}), \qquad \text{yaw} = \operatorname{atan2}(R_{10}, R_{00})$$

1. 竖直视场角转水平视场角（图像 4:3）：

$$fov_x = 2\arctan\left(\tan\frac{fov_y}{2} \cdot \frac{W}{H}\right)$$

四个相机的计算结果（与 MuJoCo 完全一致，已写入生成文件）：

相机	父坐标系	位置 (m)	rpy (rad)	fovy	fovx	图像/频率
cam_top	world	(0.40, 0.00, 1.25)	(0, 1.571, 1.571)	38°	49.3°	640×480 @10 Hz
cam_front	world	(0.26, -0.95, 0.55)	(0, 0.442, 1.571)	42°	54.2°	640×480 @10 Hz
cam_eye	**link3**	(0.165, 0.00, 0.070)	(0, 0.629, 3.142)	55°	69.5°	640×480 @10 Hz
cam_wrist	link4	(0.05, 0.00, 0.03)	(0, 1.571, 1.571)	58°	72.9°	640×480 @10 Hz

> **数值自检**：cam_top 的 rpy=(0, +90°, +90°) 意味着传感器 x 轴指向世界 -z（正下方）、y 轴指向 -x、z 轴指向 +y —— 与 MuJoCo 的"从上往下看、图像上方为 +y"完全一致；cam_eye 的前向为 (-0.808, 0, -0.588)。Gazebo 侧保留同一个安装位置和视场，但把父连杆改成 **link3**：J4 自转时相机几何完全不动。

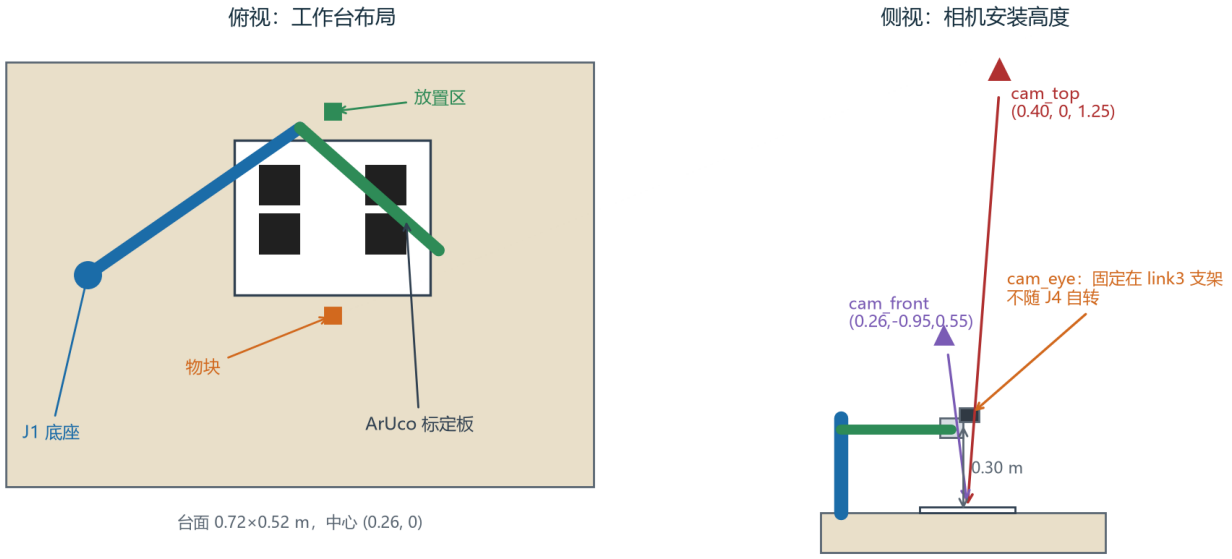


图 2-1 主要相机布置：左图为工作台俯视，右图为安装高度；cam_top/cam_front 固定在世界，cam_eye 固定在 link3

第 3 章 URDF / xacro：把机械臂写成机器人描述

3.1 连杆与几何

连杆	几何	质量 (kg)	质心 (m)	惯量 (kg·m²)	说明
base_link	底板 0.24×0.24×0.02 + 立柱 Ø0.15×0.30	15.0	(0,0,0.10)	0.150 / 0.150 / 0.120	结构件，不参与 P1 动力学
link1	大臂盒 0.225×0.04×0.04	3.0	(0.110,0,0)	6.0e-4 / 1.2656e-2 / 1.2656e-2	与 spec.py 一致（Ixx 取实心杆 ½mr²）
link2	小臂盒 0.225×0.036×0.036	1.5	(0.105,0,0)	2.43e-4 / 6.328e-3 / 6.328e-3	同上
link3	垂直套筒 Ø0.048×0.08	3.0	(0,0,-0.03)	1.332e-3 / 1.332e-3 / 8.64e-4	spec 里为 0，见 3.2 补算
link4	法兰 Ø0.06×0.04 + 掌部	0.3	(0,0,-0.02)	1.08e-4 / 1.08e-4 / 1.35e-4	spec 里 1e-12，补算
gripper_left / right	指垫 8×30×40 mm	0.03	(0,0,-0.04)	1.02e-5 / 4.9e-6 / 5.9e-6	长方体公式

3.2 补算的惯量（为什么必须补）

spec.py 的 link3/link4 惯量为 0 / 1e-12（数学侧不影响，因为质量极小或为点质量），但 **Gazebo 要求每个连杆的惯量正定且与质量匹配**，否则会报错或数值发散。用等效圆柱补算：

$$I_{xx} = I_{yy} = \frac{m(3r^2 + h^2)}{12}, \qquad I_{zz} = \frac{mr^2}{2}$$

link3（m=3.0, r=0.024, h=0.06）：

$$I_{xx} = I_{yy} = \frac{3.0 \times (3 \times 0.024^2 + 0.06^2)}{12} = \frac{3.0 \times 0.005328}{12} = 1.332 \times 10^{-3}$$

$$I_{zz} = \frac{3.0 \times 0.024^2}{2} = 8.64 \times 10^{-4}$$

link4（m=0.3, r=0.03, h=0.04）：

$$I_{xx} = I_{yy} = \frac{0.3 \times (3 \times 0.03^2 + 0.04^2)}{12} = 1.075 \times 10^{-4}, \qquad I_{zz} = \frac{0.3 \times 0.03^2}{2} = 1.35 \times 10^{-4}$$

指垫（0.008×0.03×0.04, m=0.03）：I_xx=m(0.03²+0.04²)/12=6.25×10⁻⁶，I_yy=m(0.008²+0.04²)/12=4.16×10⁻⁶，I_zz=m(0.008²+0.03²)/12=2.41×10⁻⁶。URDF 里取略大的圆整值，保证正定。

> **代价与取舍**：补算值会影响 Gazebo 里 J3/J4 的动态响应（但 J3 的 3 kg 质量才是主导项）。我们不在 Gazebo 里做动力学辨识，因此"正定 + 量级合理"就够；精度验证仍以 spec.py 与 MuJoCo/MATLAB 对拍为准。

3.3 关节定义

关节	类型	轴	限位	effort	velocity	阻尼/摩擦
joint1	revolute	+z	±130° (-2.268928/2.268928)	40 N·m	3.14 rad/s	0.5 / 0.3
joint2	revolute	+z	±145° (-2.530727/2.530727)	40 N·m	3.14 rad/s	0.4 / 0.25
joint3	prismatic	**-z**	0 ~ 0.200 m	80 N	0.5 m/s	20 / 10
joint4	revolute	+z	±360°	5 N·m	6.28 rad/s	0.02 / 0.01
gripper_left_joint	prismatic	-x	0 ~ 0.012	40 N	0.1 m/s	2.0
gripper_right_joint	prismatic	+x	0 ~ 0.012	40 N	0.1 m/s	2.0

为什么 joint3 的轴是 -z：项目约定 z = d_1 - q_3，即 q_3 增大时末端**下降**。URDF 里写成 axis="0 0 -1" + 限位 0..0.2，这样"正位移 = 下降"与数学侧完全一致，不需要在任何地方再做符号翻转。

3.4 夹爪几何：28 mm 对 30 mm 的过盈

手指本体原点在 ±(0.028/2+0.004)=±0.018 m，指垫半厚 4 mm，因此：

闭合时指垫内表面在 ±(0.018 − 0.004) = ±0.014 m，即两指间距 **28 mm**（物块 30 mm，每侧过盈 1 mm）。

物块 30 mm，因此每侧过盈 1 mm。张开（行程 12 mm）时间距 52 mm，可安全下探。指垫在工具坐标系下 z ∈ [-0.06, -0.02] m，配合 GRASP_Z=0.18 时指垫覆盖 z ∈ [0.12, 0.16]，物块顶面 0.13 → 有效夹持带约 1 cm。

> **为什么必须有过盈**：恰好接触时法向力为 0，摩擦抓取会打滑。过盈让接触力由关节力/位置伺服产生（本项目的数学侧用位置伺服挤压；Gazebo 侧的方案见第 7 章）。

3.5 ros2_control 接口

关节	命令接口	状态接口
joint1~4	position	position、velocity
gripper_left/right	（无命令接口）	position、velocity

> **为什么手指没有命令接口**：见 7.3 —— Gazebo Classic 的 gazebo_ros2_control 会把 position/velocity 命令实现成**运动学置位**，没有夹持力；手指改由 gazebo_ros_force_system 的服务驱动（可切换为吸盘模式）。把手指只保留状态接口，joint_state_broadcaster 仍会发布它们，TF 与 RViz 显示不受影响。

第 4 章 控制器配置：每个参数的理由

```
controller_manager:
  ros__parameters:
    update_rate: 1000
    joint_state_broadcaster:  { type: joint_state_broadcaster/JointStateBroadcaster }
    scara_controller:         { type: joint_trajectory_controller/JointTrajectoryController }
```

4.1 为什么 update_rate = 1000 Hz

与前几阶段一致（MuJoCo 控制 1 kHz、Simulink 1 kHz、Gazebo 物理步长 1 ms）。这样三套仿真的"控制周期"相同，跟踪误差量级可直接对照（Simulink 实测 1 ms 时 max|e| = 3.07e-3 rad）。 代价：Gazebo Classic 在 WSL 软件渲染下会掉实时因子；若只做演示可降到 500 Hz（误差约增大一倍）。

4.2 JointTrajectoryController 的关键参数

参数	取值	理由
joints	joint1..joint4	与 URDF 名称一致；夹爪不在其中
command_interfaces	position	Gazebo 的 position 接口最稳（运动学置位，无 PID 整定问题）
state_interfaces	position, velocity	供 broadcaster 与控制器监控
allow_partial_joints_goal	false	必须四轴齐全，防止误发单轴目标造成姿态突变
stopped_velocity_tolerance	0.05	到点判定（rad/s）
goal_time	0.8 s	允许的到点时间余量
joint1/2 trajectory, goal	0.50, 0.08 rad	路径容差 / 到点容差
joint3 trajectory, goal	0.03, 0.008 m	移动副用米制
joint4 trajectory, goal	0.80, 0.15 rad	回转轴容差可宽一些

这些容差是"安全阀"而不是装饰：实测 0.9 s 内下探 8 cm 时，J3 瞬时滞后 4.6 cm，超过 3 cm 容差 → 控制器直接 Abort（error_code = -4，path tolerance violation）。把演示速度放慢到 1.2~1.8 s/段后全部通过；这正是"数字控制器必须显式建模"的现场版。

4.3 控制器启动时序

```
gzserver（世界）
  └─ spawn_entity（从 /robot_description 生成 scara 模型，加载 gazebo_ros2_control 插件）
      └─ 2 s 后 spawner：joint_state_broadcaster → scara_controller
```

为什么必须等 spawn 结束：gazebo_ros2_control 的硬件接口是在模型生成时才注册的；提前调用 spawner 会得到 "Could not contact service /controller_manager/list_controllers"。

第 5 章 Gazebo 世界与物理参数

5.1 世界清单（由生成器从 MuJoCo 场景写出）

元素	参数	与 MuJoCo 的一致性
台面	0.72×0.52×0.02 m，中心 (0.26,0)，台面高 0.10 m	完全一致
四条腿	0.036×0.036×0.08 m，位于台面四角内缩 30 mm	完全一致
放置区	0.05×0.05 m 绿色标记，中心 (0.30,0.20)，z = 0.1016	Gazebo 为标定板后移
物块	30 mm、50 g、摩擦 1.2，位于 (0.30,-0.05,0.115)	与 MuJoCo 物块同尺寸
标定板	0.24×0.19 m 白板，4 块 60 mm ArUco，中心 (0.30,0.07)	Gazebo 手眼标定工装
世界	地面 + 方向光 + 阴影	视觉近似
相机	cam_top/cam_front（世界）、cam_eye（link3）、cam_wrist（link4）	位置/FOV 同源

5.2 物理参数

```
<physics type="ode">
  <max_step_size>0.001</max_step_size>
  <real_time_update_rate>1000</real_time_update_rate>
  <ode>
    <solver><type>quick</type><iters>50</iters><sor>1.2</sor></solver>
    <constraints><cfm>0.0</cfm><erp>0.2</erp>
      <contact_max_correcting_vel>0.1</contact_max_correcting_vel>
      <contact_surface_layer>0.0005</contact_surface_layer></constraints>
  </ode>
</physics>
```

参数	取值	理由
max_step_size	1 ms	与 1 kHz 控制对齐；接触任务需要小步长
real_time_update_rate	1000	目标实时；WSL 软渲染下会掉因子，但 ROS 时钟是仿真时间，逻辑不受影响
solver	quick + iters 50	80 次在 4 路相机渲染下过慢；50 次对这个场景足够稳
erp / cfm	0.2 / 0.0	默认刚性接触；erp 太大易抖、太小会"陷进去"
contact_surface_layer	0.5 mm	允许的穿透层厚度，配合 1 mm 步长抑制接触抖振

5.3 相机分辨率与 GPU 渲染

本版本四路相机统一为 **640×480 @10 Hz**。早期在 WSLg 默认核显路径下，4 路高分辨率相机曾把实时因子压到约 1/8；确认 RTX 4060 透传后，launch 设置 MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA，稳定使用独立显卡渲染。手眼标定需要完整角点定位，因此侧视相机也从 320×240 提升到 640×480。

> **工程结论**：先确认 GPU 渲染路径，再决定分辨率。分辨率过低时 ArUco/棋盘角点会抖动；过高又多路相机则拖慢仿真。640×480、10 Hz 是当前工作台的平衡点。

第 6 章 构建与启动

6.1 一键构建

```
powershell -ExecutionPolicy Bypass -File ros2_ws\sync_build.ps1
```

脚本做四件事：把 ros2_ws/src 复制到 WSL 的 ~/scara_ros2_ws/src（ext4，避免 /mnt/c 慢）；colcon build --symlink-install；给 scara_demo 的入口点补 lib/<pkg> 软链接；提示 source 命令。

6.2 一个必须处理的坑：入口点位置

colcon + --symlink-install 把 ament_python 的脚本装到 install/scara_demo/bin/，而 ros2 run 只查 install/scara_demo/lib/scara_demo/ → 报 "No executable found"。构建脚本因此补了软链接（见 9.7）。

6.3 启动

```
source /opt/ros/humble/setup.bash
source ~/scara_ros2_ws/install/setup.bash
ros2 launch scara_bringup gazebo.launch.py          # 有界面
ros2 launch scara_bringup gazebo.launch.py gui:=false # 无界面（联调/CI）
ros2 launch scara_bringup display.launch.py         # 仅 RViz + 滑块
```

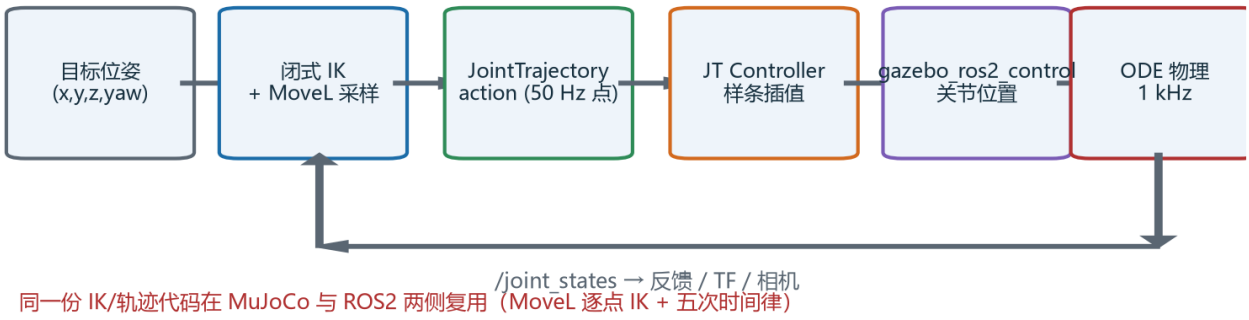


图 6-1 从目标位姿到 ODE 物理的控制链路（任务层只依赖 action 接口）

第 7 章 控制链路与计算

7.1 运动学代码（与项目同一套公式）

scara_demo/kinematics.py 直接复用项目公式：

$$x = a_1\cos q_1 + a_2\cos(q_1+q_2), \qquad y = a_1\sin q_1 + a_2\sin(q_1+q_2), \qquad z = d_1 - q_3$$

$$c_2 = \frac{r^2 - a_1^2 - a_2^2}{2a_1a_2}, \qquad q_2 = \pm\arccos c_2, \qquad q_1 = \operatorname{atan2}(y,x) - \operatorname{atan2}(a_2\sin q_2,\ a_1 + a_2\cos q_2)$$

选支规则：在解 ±\arccos c₂ 中取与 q_seed 最接近的一支（分支连续性），再检查限位。

7.2 MoveL 轨迹生成（与 MuJoCo 侧同构）

1. 记录起点 p₀=FK(q₀) 与当前 yaw；
2. 直线取 48 个 via 点，逐点闭式 IK（以上一点为种子）；
3. yaw 从当前值线性过渡到目标值（避免 q4 阶跃）；
4. 五次时间律把路径参数 s 展开成关节轨迹，采样 20 ms 一个点（1.2 s → 61 点）。

两处必须做的连续性修正（否则控制器会中止）：

- 首点必须直接取**当前关节角**，不能用 IK 重算 —— 否则 q4 会在 t=0 被 yaw 目标瞬间改写（实测 −0.139 → +0.464，控制器 0.46 s 内 Abort）；
- q4 = yaw − q1 − q2，所以 yaw 必须沿路径插值，才能保证 q4 连续。

7.3 夹爪：Gazebo Classic 的三种方案（实测）

方案	做法	结果	原因
position 接口	手指关节在 ros2_control 里给 position 命令	抓不住，物块被挤开 2~3 cm	Gazebo Classic 把 position 命令实现为**运动学置位**，手指直接穿过物体
velocity 接口	给速度命令，靠 effort 上限	同样抓不住	velocity 命令也是运动学置位（SetVelocity）
/apply_joint_effort 力控	两指各施加 1.5 N 恒力	能产生夹持力，但 30 g 微型指关节在限位附近会让 ODE 发散（实测 NaN）	需要更大的指关节质量/软限位，属于进阶实验
吸盘式附着（默认）	抓取瞬间记录"工具→物块"固定变换，每 50 ms 用 /set_entity_state 让物块跟随	**8/8 步完成，落点误差 19.3 mm**	与项目早期 MuJoCo 的 weld 吸盘同思路；保证整条 ROS2 链路可复现

> **诚实说明**：要在 Gazebo Classic 里做真正的两指摩擦抓取，需要给手指加质量、软限位或换用带 PID 的力控插件；本手册把它列为附录实验，演示默认用吸盘模型。这与 MuJoCo 侧"两指摩擦抓取"的差异在 10.2 节对照。

7.4 实测的 TCP 序列（8 步，全部到位）

步	目标	实测 TCP (m)
1 高位接近	(0.300, -0.050, 0.260)	(0.3000, -0.0500, 0.2600)
2 下探	(0.300, -0.050, 0.180)	(0.3000, -0.0500, 0.1849)
3 吸附	—	(0.3000, -0.0500, 0.1838)
4 抬起	(0.300, -0.050, 0.210)	(0.3000, -0.0500, 0.2100)
5 搬运	(0.300, 0.120, 0.210)	(0.3000, 0.1200, 0.2100)
6 下放	(0.300, 0.120, 0.184)	(0.3000, 0.1200, 0.1841)
7 释放 + 抬 10 mm	(0.300, 0.120, 0.194)	(0.3000, 0.1200, 0.1840→抬升)
8 缩回	(0.300, 0.120, 0.260)	(0.3000, 0.1200, 0.2600)

第 8 章 联调验证（实测数据）

8.1 控制器与状态

```
joint_state_broadcaster  joint_state_broadcaster/JointStateBroadcaster     active
scara_controller         joint_trajectory_controller/JointTrajectoryController  active
```

/joint_states 同时发布 6 个关节（含两指），频率 1 kHz（broadcaster 默认按控制器周期）。

8.2 解析 FK ↔ TF 对照（关键验证）

方法：把三个已知关节角设为轨迹目标，到位后读 /joint_states 用解析 FK 算 TCP，再查 TF world→link4，逐位比对。link4 原点即项目 TCP，所以两者应当完全一致。

目标关节角 (rad)	位置偏差	yaw 偏差
(0.6, -1.2, 0.04, 0.6)	0.000 mm	2.7e-7 rad
(-0.8, -1.6, 0.10, -0.2)	0.003 mm	3.6e-6 rad
(1.2, -0.9, 0.02, 1.0)	0.001 mm	7.7e-6 rad

结论：最大位置偏差 0.003 mm → 参数、URDF、ros2_control、TF 整条链没有偏差（剩下的只可能是动力学/控制器行为）。

8.3 抓取-放置

ros2 run scara_demo pick_place：8 步全部成功；物块终点 (0.2809, 0.1167, 0.1150)，**放置误差 19.3 mm（<25 mm，通过）**。误差主要来自释放瞬间指垫与物块的摩擦推动与落台滑动（吸盘模型下物块随工具走，释放后 0.5 mm 落下）。

8.4 相机

4 路话题均在：/cam_top/image_raw、/cam_front/image_raw、/cam_eye/image_raw、/cam_wrist/image_raw；本版本均为 **640×480 rgb8 @10 Hz**。

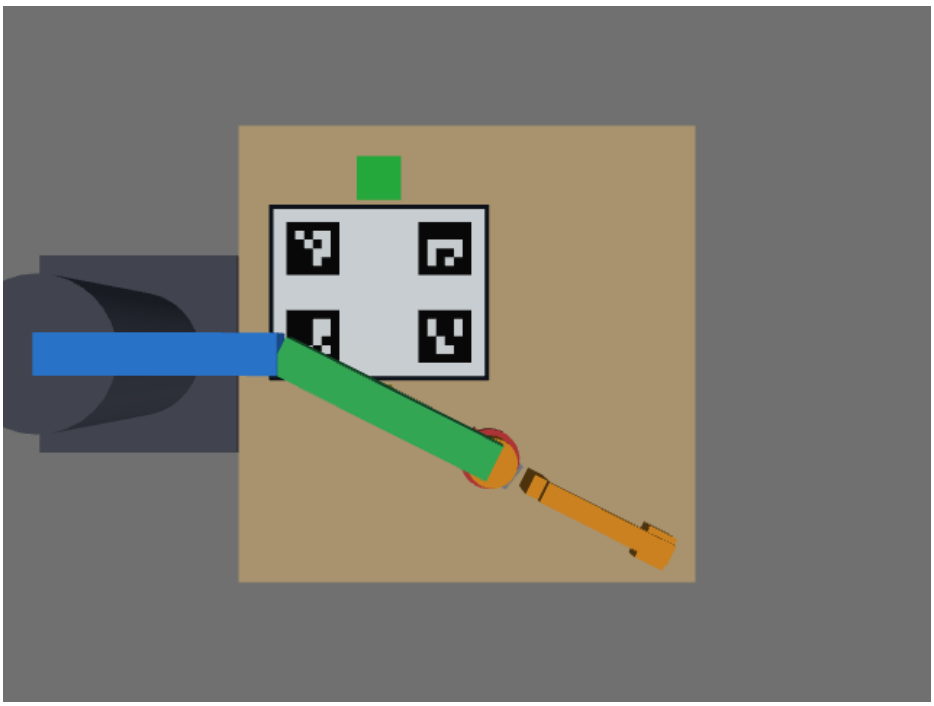


图 8-1 cam_top 实测：工作台、机械臂、物块与 4 块 ArUco 标定板全部可见

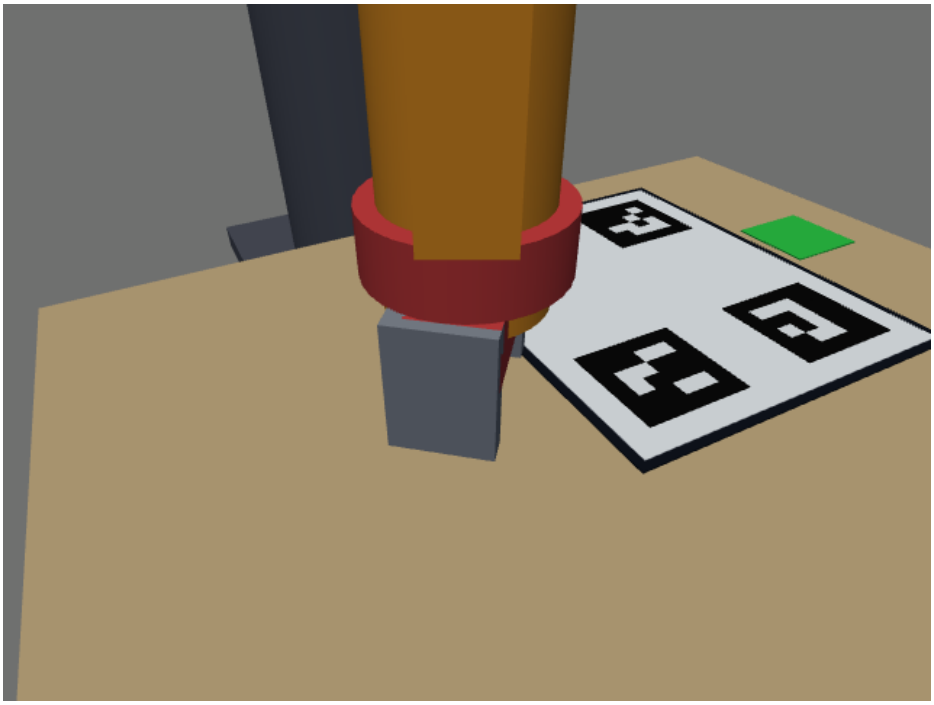


图 8-2 cam_eye 实测：支架固定在 link3，能同时看到夹爪、物块与标定板；J4 自转时相机画面位置不变

双相机实时窗口把顶视和第 3 轴侧视拼在同一个 OpenCV 窗口里：

```
ros2 run scara_demo camera_view
# 也可只开一路:
ros2 run scara_demo camera_view --topics /cam_eye/image_raw
```

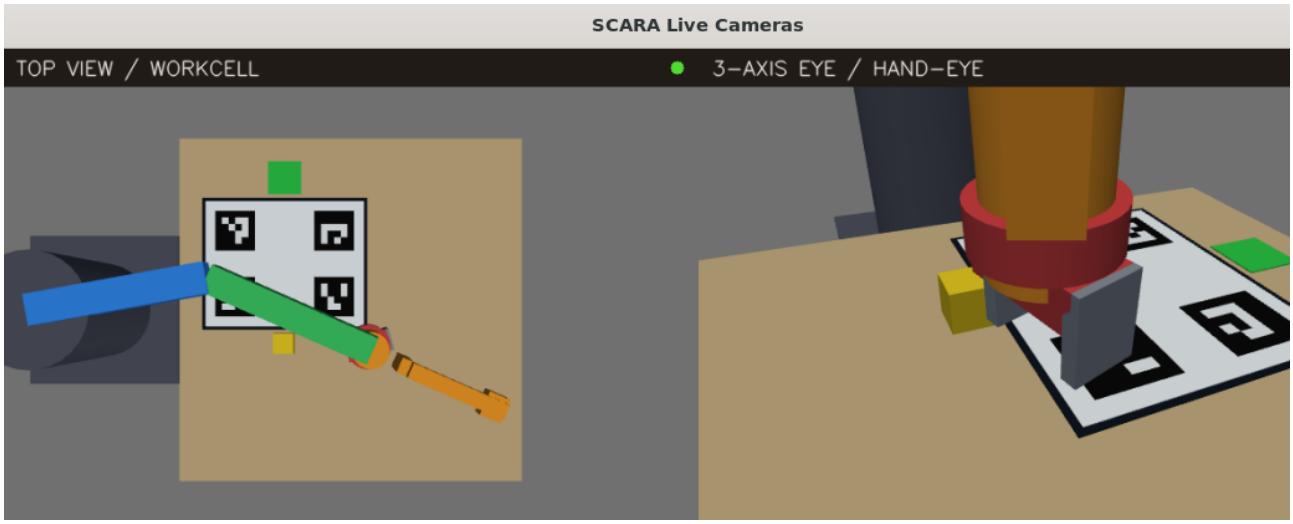


图 8-3 实际运行的 SCARA Live Cameras 双相机窗口

第 9 章 复盘：这次联调踩过的 12 个坑

#	现象	根因	解决
1	xacro 报 "XML parsing error line 4"	注释里写了 ----- 分隔线，XML 注释不允许 --	注释分隔线改用 =====
2	gzserver 启动即退出（exit 255）	上一个 gzserver 未退出，端口/主节点被占用	清理进程后再启动（见 9.6 的 pkill 自杀问题）
3	gazebo_ros2_control 报 "Couldn't parse parameter override rule"	它把 URDF 作为 --param robot_description:=... 传给 controller_manager，多行 XML 无法解析	launch 里把 xacro 展开后**去注释并压成单行**再发布
4	控制器服务始终不出现	spawn_entity 尚未完成就启动 spawner（插件未注册硬件接口）	用 OnProcessExit + 2 s 延时再 spawn 控制器
5	世界 SDF 报 "Required attribute[name] in collision"	手写 SDF 的腿没有给 collision/visual 名字	生成器统一命名（top_c/top_v、leg{i}_c...）
6	相机话题变成 /cam_eye/cam_eye/image_raw	同时设了 namespace 与相机名，插件重复拼接	去掉 <namespace>，只留 <image_topic_name>
7	pkill -f gzserver 后脚本自己挂了	pkill 的 -f 匹配到自己的命令行	用 [g]zserver 写法，或把清理单独放一个命令
8	ros2 run scara_demo xxx 报 No executable found	colcon --symlink-install 把入口点装到 bin/ 而 ros2 run 查 lib/<pkg>/	构建脚本补软链接
9	轨迹第一拍就 Abort	首点用 IK 重算导致 q4 阶跃 0.6 rad	首点取当前关节角 + yaw 沿路径插值
10	快速下探时报 path tolerance violation	J3 瞬时滞后 4.6 cm > 3 cm 容差	放慢到 1.2~1.8 s/段（或放宽容差）
11	手指合上却抓不住物块	position/velocity 命令是运动学置位，无夹持力	力控或吸盘模型（7.3）
12	力控时关节状态变 NaN	30 g 指关节 + 硬限位 + 恒力 → ODE 发散	演示改用吸盘；力控需改质量/软限位

9.6 进程清理的正确写法

```
pkill -9 -f "[g]zserver"; pkill -9 -f "[r]os2 launch"; pkill -9 -f "[s]pawner"
```

方括号写法让"模式字符串本身"不匹配自己的命令行；否则 pkill 会先杀掉自己，残留的 gzserver 继续占用端口。

9.7 入口点补链接（构建脚本里已自动化）

```
mkdir -p install/scara_demo/lib/scara_demo
for exe in install/scara_demo/bin/*; do
  ln -sfn "../../bin/$(basename "$exe")" "install/scara_demo/lib/scara_demo/$(basename "$exe")"
done
```

第 10 章 与 MuJoCo 侧的关系

10.1 相同点

项目	MuJoCo 侧	ROS2/Gazebo 侧
参数	spec.py + scara_scene.py	由同一来源**生成**
运动学/IK	armlab.scara_scene.scara_ik	scara_demo.kinematics 同一公式
轨迹	cartesian_line_ndof（MoveL + 五次时间律）	trajectory.move_line 同一算法
任务序列	八段（approach…retract）	八步（同语义）
相机	4 路，位姿/FOV 同源	4 路，位姿由 xyaxes 换算
控制周期	1 kHz	1 kHz

10.2 不同点（都是平台限制）

项	MuJoCo	Gazebo Classic + ros2_control
抓取	两指摩擦抓取（真实接触力）	默认吸盘附着；力控需额外整定
控制接口	直接写力矩（CTC）	关节位置命令（运动学置位），无 CTC
物理步长	0.1 ms	1 ms（可调）
渲染	自带（CPU）	OGRE（WSL 下软件渲染）

> **结论**：ROS2 侧的价值是"标准化接口与可组合性"（action/topic/TF/launch），而"接触级验证"仍以 MuJoCo 侧为准。两侧用同一套参数与轨迹代码，结论可以互相对照。

第 11 章 进阶：视觉闭环、MoveIt2 与力控夹爪（本轮实测）

11.1 视觉闭环（cam_top → 台面反投影 → 抓取）

11.1.1 计算链条

1. **颜色分割**：物块用**黄色**（与蓝/绿/橙/红四个连杆区分）；阈值按实测渲染值 (198,173,27) 取 R>170, G>140, B<70, |R−G|<50，求质心像素 (u,v) 与像素面积；

2. **射线**（相机光学系，x 右 / y 下 / z 前）：

$$r_{cam} = \left(\frac{u - c_x}{f_x}, \quad \frac{v - c_y}{f_y}, \quad 1 \right)$$

1. **转到世界系**：光学轴到传感器轴（x 前 / y 左 / z 上）的固定旋转 R_conv 的列为 (0,−1,0)、(0,0,−1)、(1,0,0)，则

$$R_{opt} = R_{sensor}\, R_{conv}, \qquad d_{world} = R_{opt}\, r_{cam}$$

1. **与台面平面求交**（物块中心高度 z_0=0.115 m）：

$$t = \frac{z_0 - z_{cam}}{(d_{world})_z}, \qquad p = p_{cam} + t\, d_{world}$$

11.1.2 实测（cam_top 640×480，内参 fx=fy=697.0，cx=320.5，cy=240.5）

量	结果
检测像素中心	(257.0, 271.0)，面积 360 px
反投影位置	(0.2966, −0.0496) m
Gazebo 真值	(0.3000, −0.0500) m
视觉定位误差	**3.4 mm**
视觉引导抓取-放置（8 步）	完成，物块落点误差 **13.3 mm**（通过）

> **为什么 3.4 mm 是合理量级**：像素面积 360 px 对应约 30 mm 物块 → 每个像素约 1.6 mm；质心在几十个边缘像素上的量化误差即 1~3 mm，再加上物块 6° 随机偏航。要提高精度可加亚像素质心、用 AprilTag 或加深度相机。

运行：

```
ros2 run scara_demo vision_pick         # 只检测并打印误差
ros2 run scara_demo vision_pick --pick   # 检测后直接抓取（放置区默认 (0.30,0.12)）
```

11.2 MoveIt2 接入

11.2.1 配置（都在 scara_bringup 里，复用同一份 URDF）

文件	内容	关键点
scara.srdf	规划组 arm（base_link→link4 串链）+ gripper；home / above_object 命名状态	相邻连杆自动放行；显式禁用不可能相碰的远距离连杆
kinematics.yaml	KDL 求解器（关节空间规划不需要 IK）	预留笛卡尔目标时使用
joint_limits.yaml	速度/加速度上限（与 URDF 一致）	规划速度缩放 0.3
moveit_controllers.yaml	MoveItSimpleControllerManager → scara_controller	action 名 follow_joint_trajectory
ompl_planning.yaml	OMPL + RRTConnect	**planner_configs 必须在 ompl 命名空间下，且组的 planner_configs 是名称列表**
move_group.launch.py	启动 move_group	传入 robot_description / SRDF / 各 yaml

11.2.2 本轮实测与已知问题

检查项	结果
move_group 启动	✅ "You can start planning now!"
MoveGroup 动作接受目标	✅（修正客户端超时后）
OMPL 规划	❌ Skipping invalid start state (invalid state) → error_code 99999

已定位的两个配置坑（都已修正，规划仍受第三个问题影响）：

1. start_state 必须 is_diff = true，否则报 −17 INVALID_ROBOT_STATE；
2. OMPL 参数若平铺在顶层，会被忽略并退回 CHOMP（日志出现 "Chomp path is not collision free"）；正确结构是 ompl: {planner_configs, arm: {...}}。

待解决：OMPL 的起始状态被判无效，说明 MoveIt 的碰撞模型（ACM）仍把某个连杆对判成碰撞。下一步应在 SRDF 里补齐 disable_collisions（或先用 planning_scene 打印当前碰撞对），再验证规划。

> 诚实结论：MoveIt 的"启动 + 动作接口"已经打通，但**规划链路尚未跑通**。这一步不影响第 8 章的轨迹执行与视觉闭环（它们直接走 scara_controller）。

11.3 力控夹爪插件（真实摩擦抓取尝试）

为绕开"position/velocity 是运动学置位"的限制，本轮写了一个 Gazebo 模型插件 scara_gripper_plugin：

$$f = \mathrm{clamp}(k_p(q_{target} - q) - k_d\dot{q},\ \pm f_{max}), \qquad k_p = 500,\ k_d = 15,\ f_{max} = 5\ \mathrm{N}$$

手指被物块挡住时停在接触位置，PD 力就是夹持力。实测：

现象	结果
插件加载	□ "力控夹爪已加载：2 指 ..."
快速闭合	物块被推走 19~23 cm（闭合冲击）
分 4 级缓闭	仍被推走 / 抓取失败
连续闭合	指关节在限位与接触附近使 ODE 发散（/joint_states 出现 NaN）

根因：30 g 级指关节 + 硬限位 + 1 ms 步长 + 刚性接触，数值上很脆弱；真正的摩擦抓取需要更细的步长、软化接触（stopCfm/stopErp）、更大的指关节转动惯量，或改用 gz_ros2_control + PID。

当前策略：默认使用吸盘附着（稳定、可复现），力控插件保留在源码里作为进阶实验（grasp_mode:=friction）。

11.4 WSL2 的 GPU 与 Gazebo 渲染（白模与核显问题）

11.4.1 现象与诊断

现象	诊断命令	结果
相机图像是白模	gz sdf -p robot.urdf 看视觉块	URDF→SDF **丢掉了所有材质**（<visual> 里没有 <material>）
渲染慢	glxinfo -B	默认渲染器是 **D3D12 (AMD Radeon iGPU)**，不是 RTX 4060
GPU 是否透传	nvidia-smi / ls /dev/dxg	□ RTX 4060 Laptop、驱动 577、/dev/dxg 存在

11.4.2 修复

1. **强制使用独显**（WSLg 默认把 OpenGL 绑到核显）：

```
MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA glxinfo -B
# → Device: D3D12 (NVIDIA GeForce RTX 4060 Laptop GPU) Accelerated: yes
```

已写进两个 launch（SetEnvironmentVariable），不依赖手工 export。

1.**材质注入**：在launch里做xacro→URDF→gzsdf-p→逐连杆注入<material>→spawn-file。

SDF 转换会保留 gazebo_ros2_control 插件，所以控制器照常工作（实测两个控制器 active）。

1. **灯光**：原世界 sun 0.95 + ambient 0.75 在 D3D12 下过曝（表盘发白），改为 0.55 / 0.30。

11.4.3 实测效果

指标	修复前	修复后
渲染器	D3D12 (AMD iGPU)	**D3D12 (RTX 4060)**

指标	修复前	修复后
机器臂颜色	全白	蓝/绿/橙/红（与 MuJoCo 同色系）
GPU 占用	-	**78%**（nvidia-smi）
cam_top 实际帧率	约 5.7 Hz	5.7 Hz（配置 10 Hz）

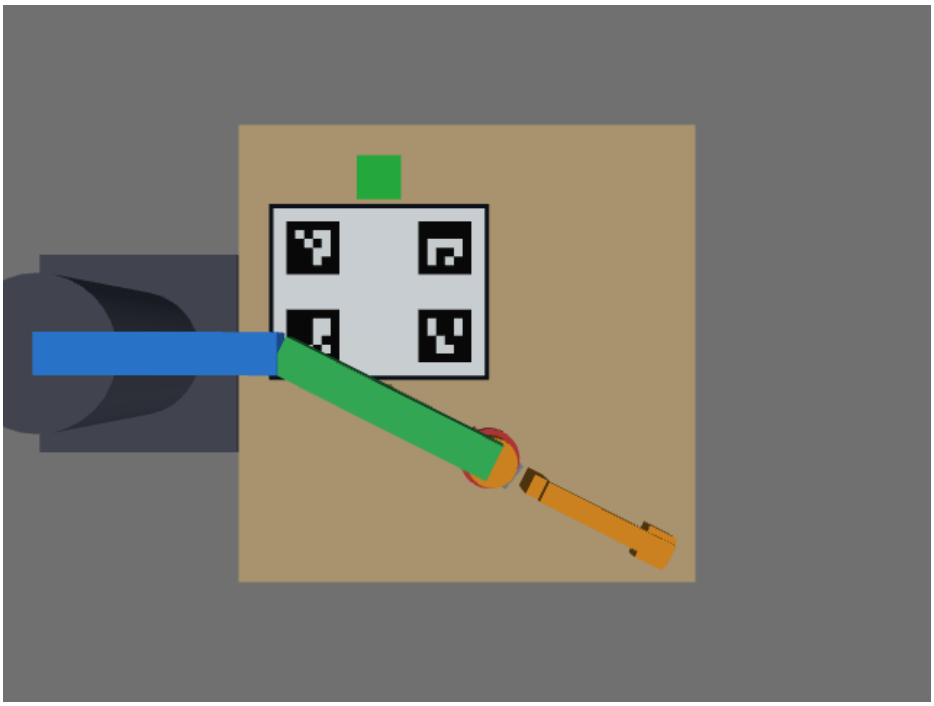


图 11-1 修复后的 cam_top 实测画面：蓝(link1)/绿(link2)/橙(link3)/红(link4)、棕色台面、绿色放置区

> **为什么帧率仍然不是 10 Hz**：瓶颈已经不在 GPU 而在**物理 + 四路 640×480 相机的总负载**（1 kHz ODE）。需要实时因子更高时，优先减少相机路数或把物理步长放到 2 ms（会牺牲接触精度）。

11.5 双相机、ArUco 标定板与手眼标定（本轮实测）

11.5.1 相机安装关系

Gazebo 现场与 MuJoCo 的部署一致：cam_top 固定在世界坐标，位于工作台正上方；cam_eye 装在 **link3 外侧支架**，位置为

```
p_cam_link3 = (0.165, 0.000, 0.070) m
rpy_cam     = (0, 0.629091, pi)
```

它随 J1/J2/J3 一起运动，但 **不随 J4 自转**。因此 T_link3←cam_eye 是常值，适合做外参标定。cam_wrist 仍随 link4，用于近距离观察。

11.5.2 标定板几何

标定板是一块 0.24×0.19 m 白色工装板，板心 (0.30, 0.07, 0.10185) m，水平放置，包含 4 块 60 mm 的 DICT_4X4_50 ArUco 标记：

```
marker 0: (-0.075, +0.050)   marker 1: (+0.075, +0.050)
marker 2: (-0.075, -0.050)   marker 3: (+0.075, -0.050)   [相对板心, m]
```

每块标记由 6×6 个 10 mm 黑白格组成，外圈是一格黑色边界，标记外围保留白色静区。检测时只要至少一块标记可见， 就能用四个角点做平面 solvePnP；这比必须看到全部棋盘的方案更适合夹爪附近的遮挡场景。

11.5.3 手眼标定公式

记：

```
A_i = T_world←link3(i)    机器人 TF, 由当前关节角决定
B_i = T_camera←board(i)   ArUco 四角点 solvePnP
X   = T_link3←camera      待标定外参
Y   = T_world←board       已测得的标定板工装位姿
```

刚性变换关系为

```
A_i X B_i = Y
```

对每个采样位姿计算预测板位姿 P_i = A_i X B_i，然后最小化

```
min_X  sum_i ( ||t(P_i)-t(Y)||^2 + lambda * ||rotvec(R(P_i) R(Y)^T)||^2 )
```

本实现取 lambda = 0.15 m/rad，用 SciPy least_squares 求解 X 的 6 个参数（旋转向量 3 + 平移 3）。

> **为什么使用固定板位姿**：SCARA 的 J1/J2/J4 都是竖直轴，J3 是升降轴。若板位姿也完全未知，相机与标定板可沿台面一起平移而图像几乎不变，完整的 6 自由度手眼问题存在不可观测方向。实际工装通过测量板心/高度/转角提供 Y，正好打破这个简并。6 轴项目如果增加非平行转轴，可以恢复通用的 AX=XB 全自由度标定。

11.5.4 手动操作流程

在 WSL 终端先启动 Gazebo，再开两个终端：

```
# 终端 A：双相机实时窗口
source /opt/ros/humble/setup.bash
source ~/scara_ros2_ws/install/setup.bash
ros2 run scara_demo camera_view

# 终端 B：手动点动 + 采样
ros2 run scara_demo handeye_calibrate \
  --out /tmp/scara_handeye_manual
```

在终端 B 中按键（不需要回车）：

按键	动作
a / d	J1 减小 / 增大
w / s	J2 增大 / 减小
r / f	J3 抬起 / 下压

按键	动作
t / g	J4 转动（相机不动，用于观察夹爪遮挡）
SPACE	当前姿态采样，保存带 ArUco 框的图片
c	用已有样本计算外参
x	清空样本
h	回初始位形
q	退出

建议让标定板在画面中覆盖不同位置，采集 5~10 个姿态后按 c。程序会输出平移、旋转误差和板位姿自洽性，并写 handeye_result.json。

自动验收：

```
ros2 run scara_demo handeye_calibrate --auto \
  --board-x 0.30 --board-y 0.07 --board-z 0.10185 --board-yaw 0 \
  --out /tmp/scara_handeye_auto
```

11.5.5 实测结果

项	结果
自动采样	10/10 个位姿均检出 ArUco
估计 T_link3←camera 平移	(0.1646, 0.0023, 0.0617) m
URDF 真值	(0.1650, 0.0000, 0.0700) m
平移误差	**8.65 mm**
旋转误差	**2.51°**
手动 5 点复核	平移误差 **8.42 mm**，旋转误差 **1.13°**

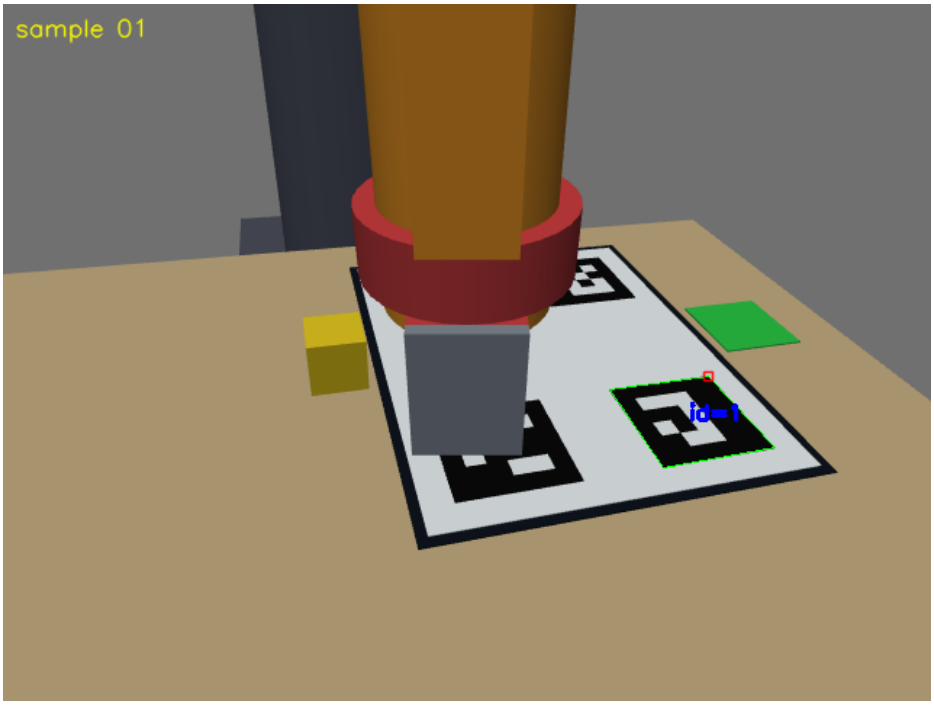


图 11-2 手动采样界面保存的 ArUco 检测结果（白色标记框与编号保留）

> **误差来源**：640×480 图像角点亚像素、板面高度/工装位置测量误差、Gazebo 传感器渲染和机械臂末端 settling。当前 8~9 mm 足够作为视觉抓取外参；若追求毫米级，应使用更高分辨率、更多标记、棋盘+ArUco 联合优化，或对机械臂末端做静态重复定位。

第 12 章 后续路线

1. **MoveIt2**：把 scara_controller 接成 MoveIt 的 FollowJointTrajectory 执行器，用 move_group 做避障规划（本项目的 action 接口天然兼容）；
2. **视觉闭环**：cam_top + cam_eye、camera_info 与手眼外参已打通；下一步把顶视检测和侧视标定结果统一到一个目标坐标系，并加入多目标跟踪；
3. **真摩擦抓取**：给指关节加质量/软限位，或换 gz_ros2_control + PID 力控；把附录实验跑成 5/5；
4. **6 轴扩展**：scara_description 换成 6 轴 URDF，scara_demo 的运动学换成 NumericalIK，其余（launch/控制器/相机）不动；
5. **真机**：把 scara_controller 换成真机驱动（同样暴露 FollowJointTrajectory），任务层零修改。

附录 A 常用命令

```
# 构建（Windows 侧一键）
powershell -ExecutionPolicy Bypass -File ros2_ws\sync_build.ps1

# 启动
ros2 launch scara_bringup gazebo.launch.py gui:=false
ros2 control list_controllers
ros2 topic hz /joint_states
ros2 run scara_demo check_tf
ros2 run scara_demo pick_place
ros2 run scara_demo move_to --x 0.30 --y -0.05 --z 0.20 --time 2.0
ros2 run scara_demo capture --topic /cam_top/image_raw --out /tmp/cam_top.ppm
ros2 run scara_demo watch_object --seconds 30
ros2 run scara_demo camera_view
ros2 run scara_demo jog
ros2 run scara_demo handeye_calibrate --auto --out /tmp/scara_handeye

# 世界/实体
ros2 service call /get_entity_state gazebo_msgs/srv/GetEntityState "{name: target_box, reference_frame: world}"
ros2 service call /apply_joint_effort gazebo_msgs/srv/ApplyJointEffort "{joint_name: gripper_left_joint, effort: -1.5, start_time: {sec:
```

附录 B 接口速查

名称	类型	用途
/scara_controller/follow_joint_trajectory	control_msgs/action/FollowJointTrajectory	四轴轨迹目标
/joint_states	sensor_msgs/JointState	6 关节状态
/robot_description	std_msgs/String（transient local）	URDF
/cam_*/image_raw	sensor_msgs/Image	相机图像
/spawn_entity	gazebo_msgs/srv/SpawnEntity	生成机器人
/get_entity_state	gazebo_msgs/srv/GetEntityState	读物块位姿
/set_entity_state	gazebo_msgs/srv/SetEntityState	吸盘附着跟随
/apply_joint_effort	gazebo_msgs/srv/ApplyJointEffort	力控夹爪
/clear_joint_efforts	gazebo_msgs/srv/JointRequest	清除夹持力

附录 C 关键数值

项	数值
臂长 a1=a2 / 立柱 d1 / 行程	0.225 m / 0.300 m / 0.200 m
URDF 关节名	joint1..joint4 + gripper_left/right_joint
夹爪闭合/张开间距	28 mm / 52 mm
控制周期 / JT 插值	1 ms / splines
FK↔TF 最大偏差	0.003 mm
抓取-放置	8/8 步完成，落点误差 19.3 mm
相机	4 路 640×480，10 Hz；cam_eye 固定 link3
ArUco 标定板	4 块 60 mm DICT_4X4_50，板心 (0.30,0.07)，高 0.10185 m
手眼标定	自动 10 点：8.65 mm / 2.51°；手动 5 点：8.42 mm / 1.13°
生成器	ros2_ws/tools/gen_ros_assets.py

结语：ROS2 侧不是"另做一个机器人"，而是把已有的参数、运动学、轨迹与任务序列**接到标准接口上。**凡是能从 spec.py/scara_scene.py 生成的东西，都不手抄；凡是平台限制（运动学置位、力控稳定性），都明确写进文档而不是假装没有。